Raport 1

Zadanie 1

Porównaliśmy output funkcji dla zapytań z wykorzystaniem funkcji agregujących oraz funkcji okna.

Funkcje agregujące

```
select avg(unitprice) avgprice
from products p;
```

avgprice
28.866363636363637

Wykorzystując funkcję `avg`, otrzymaliśmy jeden wiersz jako wynik.

```
select categoryid, avg(unitprice) avgprice
from products p
group by categoryid;
```

CategoryID	avgprice
1	37.979166666666664
2	23.0625
3	25.16
4	28.73
5	20.25
6	54.00666666666667
7	32.37
8	20.6825

grupując wg `categoryID` otrzymaliśmy liczbę wierszów równą liczbie unikalnych wartości w tej tabeli.

Funkcje okna

```
select avg(unitprice) over () as avgprice
from products p
limit 5;
```

avgprice
28.866363636363637
28.866363636363637
28.866363636363637
28.866363636363637
28.866363636363637

```
SELECT COUNT(*) FROM (
    select avg(unitprice) over (partition by categoryid) as avgprice
    from products p
);
```

COUNT(*)
77

przy użyciu funkcji over otrzymujemy liczbę wierszy równą liczbie wierszy całej tabeli.

Przykład z dwoma kolumnami lepiej obrazuje działanie agregacji przy użyciu funkcji okna

```
select categoryid, avg(unitprice) over (partition by categoryid) as avgprice
from products p;
```

▶ Wynik zapytania:

CategoryID	avgprice
1	37.979166666666664
1	37.979166666666664
1	37.979166666666664
1	37.979166666666664
1	37.979166666666664
1	37.979166666666664
1	37.979166666666664
1	37.979166666666664
1	37.979166666666664
1	37.979166666666664
1	37.979166666666664

CategoryID	avgprice
1	37.979166666666664
1	37.979166666666664
2	23.0625
2	23.0625
2	23.0625
2	23.0625
2	23.0625
2	23.0625
2	23.0625
2	23.0625
2	23.0625
2	23.0625
2	23.0625
2	23.0625
2	23.0625
3	25.16
3	25.16
3	25.16
3	25.16
3	25.16
3	25.16
3	25.16
3	25.16
3	25.16
3	25.16
3	25.16
3	25.16
3	25.16
4	28.73
4	28.73
4	28.73

CategoryID	avgprice
4	28.73
4	28.73
4	28.73
4	28.73
4	28.73
4	28.73
4	28.73
5	20.25
5	20.25
5	20.25
5	20.25
5	20.25
5	20.25
5	20.25
6	54.00666666666667
6	54.00666666666667
6	54.00666666666667
6	54.00666666666667
6	54.00666666666667
6	54.00666666666667
7	32.37
7	32.37
7	32.37
7	32.37
7	32.37
8	20.6825
8	20.6825
8	20.6825
8	20.6825
8	20.6825

CategoryID	avgprice
8	20.6825
8	20.6825
8	20.6825
8	20.6825
8	20.6825
8	20.6825
8	20.6825

Przy partycjonowaniu wg `categoryID` każdemu wierszowi zostaje przypisana średnia dla danej kategorii.

- słowo kluczowe `OVER` wskazuje na użycie funkcji okna
- `PARTITION BY` pozwala na podzielenie danych na grupy
- funkcje okna nie zmieniają liczby wierszy w tabeli, na której operują

Zadanie 2

```
select p.productid, p.ProductName, p.unitprice,
(select avg(unitprice) from products) as avgprice
from products p
where productid < 10
```

ProductID	ProductName	UnitPrice	avgprice
1	Chai	18	28.866363636363637
2	Chang	19	28.866363636363637
3	Aniseed Syrup	10	28.866363636363637
4	Chef Anton's Cajun Seasoning	22	28.866363636363637
5	Chef Anton's Gumbo Mix	21.35	28.866363636363637
6	Grandma's Boysenberry Spread	25	28.866363636363637
7	Uncle Bob's Organic Dried Pears	30	28.866363636363637
8	Northwoods Cranberry Sauce	40	28.866363636363637
9	Mishi Kobe Niku	97	28.866363636363637

```
select p.productid, p.ProductName, p.unitprice,
avg(unitprice) over () as avgprice
from products p
where productid < 10
```

ProductID	ProductName	UnitPrice	avgprice
1	Chai	18	31.372222222222224
2	Chang	19	31.372222222222224
3	Aniseed Syrup	10	31.372222222222224
4	Chef Anton's Cajun Seasoning	22	31.372222222222224
5	Chef Anton's Gumbo Mix	21.35	31.372222222222224
6	Grandma's Boysenberry Spread	25	31.372222222222224
7	Uncle Bob's Organic Dried Pears	30	31.372222222222224
8	Northwoods Cranberry Sauce	40	31.372222222222224
9	Mishi Kobe Niku	97	31.372222222222224

Dla drugiego zadania agregacja funkcją odbywa się w ramach tabeli po aplikacji filtra `WHERE productid < 10`.
W pierwszym przypadku średnia jest obliczana po wszystkich wierszach, bez działania filtra. Zatem jest ona równa średniej uzyskanej w zapytaniach z 1. zadania.

Zapytania równoważne

Pierwsze zapytaie można równoważnie zapisać jako:

```
select
    p.productid,
    p.ProductName,
    p.unitprice,
    (select avg(unitprice) over () from products) as avgprice
from products p
where productid < 10
```

▶ Wynik zapytania

ProductID	ProductName	UnitPrice	avgprice
1	Chai	18	28.866363636363637
2	Chang	19	28.866363636363637
3	Aniseed Syrup	10	28.866363636363637
4	Chef Anton's Cajun Seasoning	22	28.866363636363637
5	Chef Anton's Gumbo Mix	21.35	28.866363636363637
6	Grandma's Boysenberry Spread	25	28.866363636363637
7	Uncle Bob's Organic Dried Pears	30	28.866363636363637

ProductID	ProductName	UnitPrice	avgprice
8	Northwoods Cranberry Sauce	40	28.866363636363637
9	Mishi Kobe Niku	97	28.866363636363637

Drugie zapytanie można zapisać alternatywnie jako:

```
select
    p.productid,
    p.ProductName,
    p.unitprice,
    (
        select avg(UnitPrice)
        from Products p
        where p.ProductID < 10
    ) as avgprice
from products p
where p.ProductID < 10
```

▶ Wynik zapytania

ProductID	ProductName	UnitPrice	avgprice
1	Chai	18	31.372222222222224
2	Chang	19	31.372222222222224
3	Aniseed Syrup	10	31.372222222222224
4	Chef Anton's Cajun Seasoning	22	31.372222222222224
5	Chef Anton's Gumbo Mix	21.35	31.372222222222224
6	Grandma's Boysenberry Spread	25	31.372222222222224
7	Uncle Bob's Organic Dried Pears	30	31.372222222222224
8	Northwoods Cranberry Sauce	40	31.372222222222224
9	Mishi Kobe Niku	97	31.372222222222224

Zadanie 3

Polecenie zwracające id produktu, nazwę produktu, cenę produktu oraz średnią cenę wszystkich produktów:

Funkcja okna

```
select
    p.ProductID,
    p.ProductName,
    p.UnitPrice,
```

```
    avg(UnitPrice) over () as avgprice
from Products p
```

Podzapytanie

```
select p.ProductID,
       p.ProductName,
       p.UnitPrice,
       (select avg(UnitPrice)
        from Products p) as avgprice
from Products p;
```

Join

```
select p1.ProductID,
       p1.ProductName,
       p1.UnitPrice,
       avg(p2.UnitPrice) as avgprice
from Products p1
         join Products p2
group by p1.ProductID,
         p1.ProductName,
         p1.UnitPrice;
```

Zadanie 4

Polecenie zwracające id produktu, nazwę produktu, cenę produktu, średnią cenę produktów w kategorii, do której należy dany produkt oraz wyświetlenie tylko tych pozycji, gdzie cena jest większa niż średnia:

Funkcja okna

```
with c as (
    select p.productid,
           p.productname,
           p.unitprice,
           p.categoryid,
           avg(p.unitprice) over (partition by p.categoryid) as catavgprice    --
dzieli na grupy wedlug kategorii i liczy w niej srednia
    from products p
)
select productid,
       productname,
       unitprice,
       catavgprice
```

```
from c
where unitprice > catavgprice;
```

Podzapytanie

```
select p.productid,
       p.productname,
       p.unitprice,
       (
         select avg(p2.unitprice)
         from products p2
         where p2.categoryid = p.categoryid
       ) as catavgprice
from products p
where p.unitprice >
      (
        select avg(p2.unitprice)
        from products p2
        where p2.categoryid = p.categoryid
      );
```

Join

```
select p.productid,
       p.productname,
       p.unitprice,
       c.catavgprice
from products p
join (
    select categoryid,  --mini tabela c z category id i avgprice
           avg(unitprice) as catavgprice
    from products
    group by categoryid
) c
  on c.categoryid = p.categoryid
where p.unitprice > c.catavgprice;
```

▶ Wynik zapytań

productid	productname	unitprice	catavgprice
38	Côte de Blaye	263.5000	37.9791
43	Ipoh Coffee	46.0000	37.9791
61	Sirop d'érable	28.5000	23.0625
63	Vegie-spread	43.9000	23.0625

productid	productname	unitprice	catavgprice
6	Grandma's Boysenberry Spread	25.0000	23.0625
8	Northwoods Cranberry Sauce	40.0000	23.0625
20	Sir Rodney's Marmalade	81.0000	25.1600
26	Gumbär Gummibärchen	31.2300	25.1600
27	Schoggi Schokolade	43.9000	25.1600
62	Tarte au sucre	49.3000	25.1600
69	Gudbrandsdalsost	36.0000	28.7300
72	Mozzarella di Giovanni	34.8000	28.7300
59	Raclette Courdavault	55.0000	28.7300
60	Camembert Pierrot	34.0000	28.7300
32	Mascarpone Fabioli	32.0000	28.7300
12	Queso Manchego La Pastora	38.0000	28.7300
22	Gustaf's Knäckebröd	21.0000	20.2500
64	Wimmers gute Semmelknödel	33.2500	20.2500
56	Gnocchi di nonna Alice	38.0000	20.2500
9	Mishi Kobe Niku	97.0000	54.0066
29	Thüringer Rostbratwurst	123.7900	54.0066
28	Rössle Sauerkraut	45.6000	32.3700
51	Manjimup Dried Apples	53.0000	32.3700
10	Ikura	31.0000	20.6825
18	Carnarvon Tigers	62.5000	20.6825
30	Nord-Ost Matjeshering	25.8900	20.6825
37	Gravad lax	26.0000	20.6825

Wnioski:

- Podzapytanie jest najprostsze do zrozumienia, ale zwykle najmniej wydajne (koszt ~52%).
- Join i funkcja okna mają lepszy koszt wykonania (~24%).
- Przy małej tabeli różnice czasu są małe.

Zadanie 5

Baza Northwind została poprawnie załadowana:

count | -------+ 2310000|

Zadanie 6

Powtórka z zadania 4, ale dla zwiększonej ilości rekordów z tabelą product_history:

Funkcja okna

```
with t as (
    select id,
           productid,
           productname,
           categoryid,
           unitprice,
           avg(unitprice) over (partition by categoryid) as catavgprice
    from product_history
    where id between 1 and 5000
)
select id,
       productid,
       productname,
       categoryid,
       unitprice,
       catavgprice
from t
where unitprice > catavgprice;
```

Podzapytanie

```
with t as (
    select *
    from product_history
    where id between 1 and 5000
)
select t1.id,
       t1.productid,
       t1.productname,
       t1.categoryid,
       t1.unitprice,
       (
         select avg(t2.unitprice)
         from t t2
         where t2.categoryid = t1.categoryid
       ) as catavgprice
from t t1
where t1.unitprice >
      (
        select avg(t2.unitprice)
        from t t2
        where t2.categoryid = t1.categoryid
      );
```

Join

```
with t as (
    select *
    from product_history
    where id between 1 and 5000
),
a as (
    select categoryid,
           avg(unitprice) as catavgprice
    from t
    group by categoryid
)
select t.id,
       t.productid,
       t.productname,
       t.categoryid,
       t.unitprice,
       a.catavgprice
from t
join a
  on a.categoryid = t.categoryid
where t.unitprice > a.catavgprice;
```

▶ Wynik zapytań (fragment)

id	productid	productname	categoryid	unitprice	catavgprice
38	38	Côte de Blaye	1	95.67	28.792622
115	38	Côte de Blaye	1	239.41	28.792622
120	43	Ipoh Coffee	1	50.05	28.792622
192	38	Côte de Blaye	1	264.99	28.792622
197	43	Ipoh Coffee	1	54.51	28.792622
269	38	Côte de Blaye	1	68.97	28.792622
346	38	Côte de Blaye	1	252.71	28.792622
351	43	Ipoh Coffee	1	52.37	28.792622
423	38	Côte de Blaye	1	160.08	28.792622
428	43	Ipoh Coffee	1	36.20	28.792622
500	38	Côte de Blaye	1	174.35	28.792622
505	43	Ipoh Coffee	1	38.69	28.792622
577	38	Côte de Blaye	1	137.88	28.792622
582	43	Ipoh Coffee	1	32.32	28.792622

id	productid	productname	categoryid	unitprice	catavgprice
654	38	Côte de Blaye	1	204.84	28.792622
659	43	Ipoh Coffee	1	44.01	28.792622
731	38	Côte de Blaye	1	171.25	28.792622
736	43	Ipoh Coffee	1	38.15	28.792622
808	38	Côte de Blaye	1	125.14	28.792622
813	43	Ipoh Coffee	1	30.10	28.792622
885	38	Côte de Blaye	1	47.40	28.792622
962	38	Côte de Blaye	1	123.08	28.792622
967	43	Ipoh Coffee	1	29.74	28.792622
1039	38	Côte de Blaye	1	153.46	28.792622
1044	43	Ipoh Coffee	1	35.04	28.792622
1116	38	Côte de Blaye	1	260.07	28.792622
1121	43	Ipoh Coffee	1	53.66	28.792622
1193	38	Côte de Blaye	1	139.13	28.792622
1198	43	Ipoh Coffee	1	32.54	28.792622
1270	38	Côte de Blaye	1	183.89	28.792622
1275	43	Ipoh Coffee	1	40.36	28.792622
1424	38	Côte de Blaye	1	31.85	28.792622
1501	38	Côte de Blaye	1	162.46	28.792622
1506	43	Ipoh Coffee	1	36.62	28.792622
1578	38	Côte de Blaye	1	131.11	28.792622
1583	43	Ipoh Coffee	1	31.14	28.792622
1655	38	Côte de Blaye	1	75.31	28.792622
1732	38	Côte de Blaye	1	127.98	28.792622
1737	43	Ipoh Coffee	1	30.60	28.792622
1809	38	Côte de Blaye	1	111.97	28.792622
1886	38	Côte de Blaye	1	160.32	28.792622
1891	43	Ipoh Coffee	1	36.24	28.792622
1963	38	Côte de Blaye	1	204.58	28.792622
1968	43	Ipoh Coffee	1	43.97	28.792622

id	productid	productname	categoryid	unitprice	catavgprice
2040	38	Côte de Blaye	1	58.42	28.792622

Wnioski:

- Podzapytanie ma najwyższy koszt i czas: 38% i ~0,2s. Powodem tego jest to, iż średnia jest liczona osobno dla kolejnych wierszy.
- Join i funkcja okna wypadają lepiej, koszt 31%; czas ~0,06s. Join i okna liczą średnią raz, co skraca czas i koszt.